

Partner Portfolio) | slides (GitLab internal) | video (GitLab internal) |
| Building Pipeline with Your Partners - MDF and Marketing Campaigns | slides (GitLab internal)
| video (GitLab internal) |

Solution Architects

Session Name	Presentation	Video
Landing New Logos	slides (GitLab internal)	video (GitLab internal)
Expand Strategy Into Security and Plan	slides (GitLab internal)	video (GitLab internal)
Strategies to Win Without a Long PoV (Proof of Value)	slides (GitLab internal)	video
(GitLab internal)		
Leveling Up the Conversation with the 5 Whys	slides (GitLab internal)	video (GitLab
internal) |

Customer Success Managers

Session Name	Presentation	Video
Account Based Strategy & Marketing for Customers	slides (GitLab internal)	video (GitLab
internal)		
Expand Strategy Into Security and Plan	slides (GitLab internal)	video (GitLab internal)
Embracing Differences with Social Styles	slides (GitLab internal)	no video
Leveling Up the Conversation with the 5 Whys	slides (GitLab internal)	video (GitLab
internal) |

Professional Services Engineers

Session Name	Presentation	Video
How to Work with Partners: Tactically and Technically	slides (GitLab internal)	no video
Professional Services Methodology: Step by Step	slides (GitLab internal)	no video
Using Congregate to Migrate Customers to GitLab	slides (GitLab internal)	no video
Leveling Up the Conversation with the 5 Whys	slides (GitLab internal)	video (GitLab
internal) |

Sales Development Reps

Session Name	Presentation	Video
Thriving as a Remote SDR: From Cold-calling Jiu-jitsu to Planning Long Term Career Success		
(featuring Tito Bohrt)	no slides	video (GitLab internal)
Converting Free to Paid	slides (GitLab internal)	video (GitLab internal)
SDR Competitive Blueprint	slides (GitLab internal)	no video
Security: Our Biggest Differentiator	slides (GitLab internal)	video (GitLab internal)

Field Marketing

Session Name	Presentation	Video

Account Based Strategy & Marketing for Customers	slides (GitLab internal)	video (GitLab internal)
Field Marketing Goals for FY22	slides (GitLab internal)	no video
Building Pipeline with Your Partners - MDF and Marketing Campaigns	slides (GitLab internal)	video (GitLab internal)

SKO 2021 Wrap Up

- SKO 2021 Wrap Up presentation (public)

SECTION: sales/training/sko/2022.md

Overview

GitLab SKO 2022 took place virtually from March 15-17, 2022. The virtual event consisted of three half-days offered in two time zone options to accommodate our team members around the world. To learn more about the event, see the Field Team FAQ document.

SKO 2022 Event Promo Video

```
<figure class="videocontainer">  
  <iframe src="https://player.vimeo.com/video/673286284?h=4d4bf9f431&color=fc6d26"  
frameborder="0" allowfullscreen="true"> </iframe>  
</figure>
```

Watch Again

All recordings from SKO 2022 can be found in Highspot. If you do not have Highspot access, you can also find the recordings in the Sales Kickoff 2022 channel in EdCast (GitLab internal-only).

SKO 2022 Day 1 Sessions

Session Name	Video
Welcome Keynote	video link
Product Keynote	video link
Field Spotlight 1	video link
Customer Speaker	video link
Field Spotlight 2	video link

SKO 2022 Day 2 Sessions

Session Name	Video
Motivational Speaker	video link
Channel & Alliances Keynote	video link
Field Spotlight 3	video link

Marketing Keynote	video link
Field Spotlight 4	video link
Closing Keynote	video link

SKO Awards Ceremony

Session Name	Video
Awards Ceremony	video link

SKO 2022 Role-Based Breakout Sessions

Enterprise Sales Strategic Account Leaders

Session Name	Presentation	Video
Delivering Strategic Customer Outcomes by Partnering with Your SAs	N/A	video link
Using Effective Proposals to Advance and Win Deals	N/A	video link
Supersonic pipeline: Best practices for prospecting to 4x	N/A	video link

Commercial Sales Account Executives

Session Name	Presentation	Video
The Big Pitch	slides	video link
Capturing the "Why Now"	slides	video link
The Journey to Ultimate, Part Two	slides	video link

Channel Sales Account Managers

Session Name	Presentation	Video
Developing your Select Partners to their Maximum Potential and Value Opportunity	slides	video link
Building Pipeline with your Partners MDF and Marketing Campaigns	N/A	video link
Partnering with Alliances to Increase Revenue and Value	slides	video link

Solution Architects

Session Name	Presentation	Video
Positioning the GitLab Agent for Kubernetes in your sales deal	slides	video link
Value Stream Mapping - Origins, Assessments and Iterations	N/A	video link
GET Workshop	slides	video link

Customer Success Managers

Session Name	Presentation	Video
CSM Strategy	slides	video link

| CSM and Customer Personas | slides | video link |
| Driving Adoption With CI/CD Workshops | slides | video link |

Professional Services Engineers

Session Name	Presentation	Video
Professional Services & Product: Working Better Together	slides	video link
How to use the GitLab Pipeline COE w/ Governance Pipelines	slides	video link
GET Workshop	slides	video link

Sales Development Reps

Session Name	Presentation	Video
Competitive Workshop	slides	video link
Partnering with Sales: Good to Great	slides	video link
Alumni Stories: Lessons Learned	slides	video link

Field Marketing

Session Name	Presentation	Video
Integrated Marketing Goals	N/A	video link

SKO 2022 Wrap Up

- SKO 2022 Wrap Up presentation (public)

SECTION: sales/training/sko/2023.md

Overview

GitLab SKO 2023 took place from February 13-17, 2023. The event was offered in a hybrid format to accommodate our team members around the world. To learn more about the event, see the Field Team FAQ document.

Event Recordings

All SKO 2023 session recordings are now available! There are two ways to view all mainstage sessions, pre-recorded breakout sessions and supplemental videos:

1. Access the SKO 2023 page in Highspot
1. If you do not have access to Highspot, you can view the SKO 2023 Recordings Google Drive folder

Remember: All SKO 2023 content is SAFE and should not be downloaded or shared externally for any reason.

SECTION: sales/training/sko/_index.md

Sales Kickoff (SKO) Overview

This event sets the tone for the GitLab field organization for the new fiscal year. The objective of SKO is that team members will leave feeling...

1. Motivated: "Our product is helping our customers achieve amazing things and the vision for our market is incredible. I know we have what it takes to win. I feel energized by the opportunity to connect with team members in-person/virtually."

1. Celebrated: "I feel like my team and leaders recognize and appreciate the work I do in helping GitLab grow and succeed."

1. Enabled: "I am confident that I have the skills and resources needed to excel in my territory/role and predictably Land with vision and Expand with purpose. I feel supported by Marketing and Product teams who want to help me win."

We will achieve this by creating an event that is:

1. Immersive

1. Engaging

1. Inclusive

Sales Kickoff Planning

For more information about the Sales Kickoff planning core team and process, see the Sales Kickoff Planning page.

SECTION: sales/training/sko/sko-planning.md

Sales Kickoff Core Team

Sales Kickoff is too big to have a single DRI for everything. The Field Enablement leader is ultimately accountable for delivery of a successful, on-budget SKO and will be responsible for the following:

- Overall story arc
- SKO agenda
- CRO leader interlocks & communication
- SKO Ambassador program

See the SKO 2023 parent epic. Additional SKO responsibilities will be split into 3 sub-teams with category DRIs responsible for specific aspects of SKO as outlined below. The SKO core team will include the following sub-teams, all of which have supporting team members underneath them:

1. Event Support & Logistics DRI

- See the SKO 2023 Event Support & Logistics epic for additional details

1. Content DRI

- See the SKO 2023 Content epic for additional details

1. Communications DRI

- See the SKO 2023 Communications epic for additional details

The SKO core team also includes an executive sponsor (VP, Field Ops) with responsibilities that include:

- Champion and socialize CRO Leaders' POV in SKO planning conversations
- Ensure regular CRO leader check-ins with executives
- Help drive alignment across CRO leadership team
- Secure exec reviews (if/as needed)
- Help prioritize where exec review and approval is needed vs. when and where the team has autonomy to make decisions

SKO Sub-Team Responsibilities

| Sub-Team | Responsibilities | Key Collaborators |

| ----- | ----- | ----- |

| Event Support & Logistics | - Brand / Design
- Budget management
- Venue details
- Partner Summit logistics
- F&B (onsite & offsite)
- Registration
- Run of show
- Event check-in
- Signage
- Mobile app
- Ancillary events (if applicable) | - Brand
- Field Enablement
- Channel Programs |

| Content | - Mainstage content
- Non-mainstage content
- Partner Summit content
- Awards & Compensation
- Production Video

 see additional details in the Content Sub-Team Deeper Dive section below | see additional details in the Content Sub-Team Deeper Dive section below |

| Communications | - Comms plan
- Event copy
- Slack channels
- Survey planning & execution
- Post-event content sharing
- Event summary / wrap-up | Events |

Content Sub-Team Deeper Dive

| Content Type | Responsibilities | Key Collaborators |

| ----- | ----- | ----- |

| Mainstage | - Keynote content
- Customer and/or partner speaker sessions
- Motivational speaker | - Events
- Corp Comms
- Execs |

| Non-mainstage | - Role-based breakouts
- Birds of a Feather sessions | - Events
- Portfolio Marketing
- Field org |

| Partner Summit | - Partner Summit sessions
- Partner sponsorships | - Events
- Portfolio Marketing
- Field Enablement |

| Awards & Compensation | - Tracking final results
- Awards
- President's Club winners
- Comp Overview | - Events
- CRO leaders |

| Production Video | - Video for mainstage sessions
- Video content production for SKO and Partner Summit, Event recording, and Other | - Events
- Digital Production
- Field Enablement
- Channel |

Managing SKO Work

Scheduling and logistics are managed in a Sales Kickoff (SKO) 2023 Agenda & Logistics Google sheet, while actual work is managed in GitLab.

SKO work within GitLab is managed in epics and issues (see SKO 2023 meta epic). Sub-team DRIs are empowered to manage their child epics and issues as they see fit.

The Sales Kickoff (SKO) 2023 Agenda & Logistics Google sheet contains:

1. SKO Mainstage Agenda
1. SKO Breakout Schedule
1. Partner SKO schedule
1. Ancillary Events schedule
1. SKO Planning Timeline
1. Due Dates
1. SKO Planning GitLab Directory (links to the relevant Epics & notes docs for SKO planning for ease of use)

The sub-team DRIs are responsible for maintaining the efficacy of this document unless there is a core team member directly responsible for all of the contents of a tab (i.e. Partner Summit, Ancillary Events, Breakouts).

SKO Planning Meetings & Cadence

The SKO planning team has one 50-minute planning meeting that includes ALL DRIs and sub-DRIs + team members leading planning and content for keynote sessions.

- SKO Core Team Weekly Meeting
 - Q2: Monthly
 - Q3: Weekly
 - Q4: 2x weekly (consider making one occurrence 25 min)

In addition, the SKO planning team delivers CRO Leader Status Updates in the following cadence:

- Q3 (September 2022): SKO 2022 planning overview, high-level objectives, theme, and draft agenda discussion
- Q3 (October 2022): SKO theme confirmation and messaging hierarchy kickoff (detailed goals & objectives)
- Q4 (Nov-Jan): Bi-weekly progress reviews

SKO "Call for Content"

In order to drive relentless prioritization based on GitLab Sales Kickoff (SKO) theme/sub-themes/goals, gather content in a timely manner and secure executive consensus early, the SKO Core Team implements a "Call for Content" for SKO breakout sessions. A secondary benefit of this process is that it gives Field Enablement a strong backlog of well-rounded training and enablement topics for topic submissions not selected or prioritized for SKO (e.g. webcast topics, etc.) in future months/quarters to support continuous education initiatives for the field organization. There are two types of breakout sessions:

1. Role-Based Breakout Sessions: Teaching skills and behaviors aligned to specific role(s)

within the organization.

1. Submit a role-based breakout session idea using this issue template

1. "Choose Your Own Adventure" Breakout Sessions: Allow attendees to step out of the role-based breakout experience and select from a list of breakout topics for the final breakout. These topics can be a deep-dive into a keynote topic (ex. diving into a feature that Product discussed in their keynote) or a broader topic or soft-skill (ex. DIB in Sales, Project Management, etc.).

1. Request a topic or focus for a "Choose Your Own Adventure" breakout using this issue template

Note that we will also use the Call for Content submission form for Mainstage sessions to tease out the learning objectives and outlines, but we will not implement a formal selection process as there is already a prescriptive "formula" for which speakers/teams give keynotes.

High Level Process

Field Enablement Program Managers send a "Call for Content" for their relevant segment areas (ENT, COMM, Customer Success, Channel & Alliances) and give DRIs/team members 3 weeks to send in submissions. Submissions are then prioritized based on a set list of criteria and relevance to the SKO theme. Any sessions not chosen are put into a backlog for consideration in fast-follow enablement/reinforcement activities following SKO. Please see below for a more detailed overview of the process.

SKO Breakout "Call for Content" Process

Below are the activities that Field Enablement Managers and Sales Kickoff (SKO) DRI collaborate on as a part of the "Call for Content" for SKO breakout sessions. This process begins no later than three months before SKO.

1. Field Enablement Program Managers to meet with segment VPs and determine on what types of content (topic, format, etc.) to solicit in their Call for Content.

1. Field Enablement Program Managers open their segment's "Call for Content" and socialize it in relevant meetings/Slack channels.

1. Call for Content stays open for 3 weeks.

1. Call for Content submissions for role-based breakout sessions are made via the SKO Call for Content Submission issue template

1. Call for Content submissions for "Choose Your Own Adventure" breakout sessions are made via the SKO Call for Content Submission issue template

1. Submissions are managed on this issue board with the following labels for organization:

1. Static labels: ~field enablement ~field events ~FE priority::1 ~sko ~sko content submission

1. Scoped labels: ~sko content::triage, ~sko content::accepted, ~sko content::deferred, ~FE status::triage

1. Once Call for Content closes, Field Enablement Program Managers review the submissions and choose top submissions based on the selection process outlined below.

1. Field Enablement Managers notify DRIs of chosen submissions and begin working with them on next steps to further build out session content.

1. Label for these accepted issues changes to ~sko content::accepted and ~FE status::wip

1. Field Enablement Managers notify DRIs for submissions not chosen. Issues for all submissions that are not chosen (and are viable, i.e. not a partial/irrelevant submission) are moved into

the Field Enablement Backlog where they can be considered for upcoming enablement initiatives.

1. ~sko content::deferred and ~FE status::backlog

Selection Criteria & Process

Below is are the criteria used to select submissions for SKO breakout sessions.

Criteria

1. Content directly supports the SKO objectives
1. Proposed session is interactive and takes advantage of in-person modality
1. High content quality
1. Fresh content (not a rehash of old ideas or topics)
1. Session topic embodies the GitLab values
1. Content fits well in the landscape of all other topics being chosen (i.e. a blend of technical, functional/soft skill topics)

Selection Process

1. All Field Enablement Program Managers and the Field Enablement leader meet in a working session to review submissions for each segment and select the winning submissions based on the criteria.

1. Note: Doing this in a working session vs. silo will help the team have healthy discussions around relevance and will uncover any areas of overlap.

1. Field Enablement Program Managers review/justify selections with their segment area VPs to ensure alignment.

SECTION: security/_index.md

<i class="fas fa-rocket" id="biz-tech-icons"></i> Security Vision and Mission

Our vision is to transparently lead the world to secure outcomes.

Our mission is to enable everyone to innovate and succeed on a safe, secure, and trusted DevSecOps platform. This will be achieved through 5 security operating principles:

1. Accelerate business success with a focus on:
 - Prioritize 'boring', iterative solutions that minimize risk
 - Find ways to say Yes
 - Understand goals before recommending solutions
 - Use GitLab first
1. Efficient operations with a focus on:
 - Technical controls over handbook rules
 - Leverage automation first (robots over humans)
 - Responsible decisions (Spending, Tooling, Staffing, etc) over low ROI (return on investment) decisions
 - Reusable or repeatable over singular solutions
1. Transparency with a focus on:

- Responsible protection of MNPI (material non-public information)
 - Evangelize dogfooding of GitLab publicly
 - Lead with metrics
 - Balance security with usefulness
1. Risk Reduction with a focus on:
 - Secure by default
 - Preventative controls over detective controls
 - Solving root causes over treating symptoms
 - Visibility through Coverage, Discoverability, Observability
 1. Collaborative Culture with a focus on:
 - Working together on common solutions
 - Solve shared problems with shared solutions
 - Simplifying language for everyone to understand
 - Avoiding security jargon
 - Seek opportunities to help others succeed

To help achieve the vision of transparently leading the world to secure outcomes, the Security Division has nominated a Security Culture Committee.

Division Structure

The Security Division provides essential security operational services, is directly engaged in the development and release processes, and offers consultative and advisory services to better enable the business to function while minimising risk.

To reflect this, we have structured the Security Division around four key tenets, which drive the structure and the activities of our group. These are :

Product Security
Security Operations
Corporate Security
Security Assurance

Application Security Infrastructure Security Security Platforms and Architecture Vulnerability Management Data Security Security Logging	
Security Incident Response Team (SIRT) Trust and Safety Red Team Threat Intelligence Signals Engineering	
Corporate Security Helpdesk Support Tech Stack Systems Engineering Teams	
Field Security Security Compliance Security Governance Security Risk	

Secure the Product - The Product Security Department

The Product Security Department is primarily focused on Securing the Product. This reflects the Security Division's current efforts to be involved in the Application development and Release cycle for Security Releases, Infrastructure Security, and our HackerOne bug bounty program.

The term "Product" is interpreted broadly and includes the GitLab application itself and all other integrations and code that is developed internally to support the GitLab application for the multi-tenant SaaS. Our responsibility is to ensure all aspects of GitLab that are exposed

to customers or that host customer data are held to the highest security standards, and to be proactive and responsive to ensure world-class security in anything GitLab offers.

Protect the Company - The Security Operations Department

Security Operations Department teams are primarily focused on protecting GitLab the business and GitLab's platform. This encompasses protecting company property as well as to prevent, detect and respond to risks and events targeting the business and our platform. This department includes the Security Incident Response Team (SIRT) and the Trust and Safety team.

These functions have the responsibility of shoring up and maintaining the security posture of GitLab's platform to ensure enterprise-level security is in place to protect our new and existing customers.

Lead with Data - The Threat Management Department

Threat Management Department teams are cross-functional. They are responsible for collaborating across the Security Division to identify, communicate, and remediate threats or vulnerabilities that may impact GitLab, our Team Members or our users and the community at large.

Assure the Customer - The Security Assurance Department

The Security Assurance Department is comprised of the teams noted above. They target Customer Assurance projects among their responsibilities. This reflects the need for us to provide resources to our customers to assure them of the security and safety of GitLab as an application to use within their organisation and as an enterprise-level SaaS. This also involves providing appropriate support, services and resources to customers so that they trust GitLab as a Secure Company, as a Secure Product, and Secure SaaS

Protect the Organization - Corporate Security

GitLab is both a company and a product. The Corporate Security department focuses on implementing and protecting the information technology (IT) related systems that the company uses to conduct business internally, and provides the hardware, software, and tools that our team members and 3rd party service providers (aka contractors) need to be productive and get their job done efficiently. The configurations that we implement for team members internally are designed to protect our customers and their data.

We have a 24x5 technical support helpdesk for team members and have engineers that configure and maintain many of our company-wide tech stack applications.

We invest heavily in device trust, identity management, and infrastructure governance to provide the highest level of security assurance for the administrators of our product and ensure all appropriate controls are in place when handling customer data.

Other groups and individuals

Security Program Management

Security Program Management is responsible for complete overview and driving security

initiatives across Product, Engineering, and Business Enablement. This includes the tracking, monitoring, and influencing priority of significant security objectives, goals, and plans/roadmaps from all security sub-departments. Security Program Manager Job Family

Security Program areas of focus

- Drive Accountability & Visibility for Program Objectives & Goals
- Drive, Gather, & Examine Program Needs & Opportunities through Intra & Inter Organizational Collaboration
- Provide Insights & Suggestions Impacting Program Strategy & Roadmap
- Assist in Gathering & Prioritizing Program Risks, Requirements, & Alignment to Influence Remediation
- Drive & Define Acceptance Criteria, Value Proposition, Milestones to Visualize and Communicate Program Effectiveness
- Develop Repeatable, Scalable, Efficient, Effective, Processes & Procedures

Product development

In keeping with our core values and the belief that everyone can contribute, the Security Division is committed to dogfooding and contributing to the development of the GitLab product.

Important topics

Tokens

The following best practices will help ensure tokens are handled appropriately at GitLab. For detailed requirements regarding the use of tokens at GitLab, please see our token management standard.

1. When creating a Personal Access Token, be sure to choose the appropriate scopes that only have the permissions that are absolutely necessary.
1. Oftentimes a Project Access Token might be sufficient instead of a Personal Access Token. Project Access Tokens have a much more limited scope and should be preferred over Personal Access Tokens whenever possible.
1. Always set an expiration for your tokens when creating them. Tokens should preferably expire in a matter of hours or a day.
1. Be mindful to keep these personal access tokens secret. Be particularly careful not to accidentally commit them in configuration files, paste them into issue or merge request comments, or otherwise expose them.
1. Please consider periodically reviewing your currently active Personal Access Tokens and revoking any that are no longer needed.
1. Personal Access Tokens will be highly discouraged within the GitLab production environment, and disallowed/disabled wherever possible. Existing tokens shall remain, but additional issuance will not be permissible/possible.
1. If you believe a personal access token has been leaked, revoke it immediately (if possible) and contact the security team using the /security Slack command.

Receive notification of security releases

- To receive security release blog notifications delivered to your inbox, visit our contact us page.
- To receive release notifications via RSS, subscribe to our security release RSS feed or our RSS feed for all releases.
- For additional information regarding security releases, please visit the Delivery Team's security releases page.

<i class="fas fa-book" style="color:rgb(110,73,203)" aria-hidden="true"></i> Resources

Tools

- Incident-Tools (private)
for working scripts and other code during or while remediating an incident. If the tool is applicable outside of the GitLab.com environment, consider if it's possible to release when the ~security issue becomes non-confidential. This group can also be used for private demonstration projects for security issues.
- security-tools (mostly private) contains some operational tools used by the security teams. Contents and/or configurations require that most of these projects remain private.

Calendar

We welcome GitLab team members to join meetings that are on our shared Security Calendar.

Other Frequently Used GitLab.com Projects

Security crosses many teams in the company, so you will find ~security labeled issues across all GitLab projects, especially:

- gitlab-foss
- gitlab
- infrastructure
- production

When opening issues, please follow the Creating New Security Issues process for using labels and the confidential flag.

Other Resources for GitLab Team Members

- Security Best Practices, using 1Password and similar tools, are documented on their own security best practices page.
- Secure Coding Training.
- GitLab.com data breach notification policy.
- GitLab Internal Acceptable Use Policy.
- For GitLab.com, we have developed a Google Cloud Platform (GCP) Security Guidelines Policy document, which outlines recommended best practices, and is enforced through our security automation initiatives.
- GitLab Security Tanuki for use on security release blogs, social media and security related swag as appropriate:

- Web-RGB
- Print-CMYK
- and one exclusively for stickers.
- Security READMEs
- Working in Security
- Contributing to GitLab the product as a Security team member
- Threat Modeling

SECTION: security/contributing-to-gitlab-the-product.md

Product Security Code Contributions

Security Engineers typically act as Subject Matter Experts and advisors to GitLab's engineering teams. Security Engineers may wish to make a larger contribution to GitLab products, for example a defense-in-depth measure or new security feature.

Like any contributor, follow the Contributor and Development Docs, paying particular attention to the issue workflow, merge requests workflow, style guides, and testing standards.

Security Engineers will need to collaborate with and ultimately hand over their work to a team in the Development Department. That team will be responsible for prioritisation, review, rollout, error budget, and maintenance of the contribution. Security Engineers should ideally open an Issue or Epic as early as possible, labeled with the candidate owning team. The team can inform implementation or architectural decisions, highlight existing or upcoming work that may impact yours, and let them plan capacity for reviewing your work.

If a team does not have capacity or a desire to assist, a Security Engineer's work can still continue; everyone can contribute.

Requests from Security Engineers for new features and enhancements should follow the process in "Requesting something to be scheduled"

This does not apply to addressing security vulnerabilities or dependency updates, which have separate processes for triage and patching.

External Code Contributions

We have a process in place to conduct security reviews for externally contributed code, especially if the code functionality includes any of the following:

- Processing credentials/tokens
- Storing credentials/tokens
- Logic for privilege escalation
- Authorization logic
- User/account access controls
- Authentication mechanisms

The Security Team works with our Community Outreach Team to ensure that security reviews are

conducted where relevant. For more information about contributing, please reference the [Contribute to GitLab](#) page.

Package Signing{package-signing}

The packages we ship are signed with GPG keys, as described in the [GitLab documentation](#). The process around how to make and store the key pair in a secure manner is described in the [runbooks](#). The Distribution team is responsible for updating the package signing key. For more details that are specific to key locations and access at GitLab, find the internal google doc titled "Package Signing Keys at GitLab" on Google Drive.

SECTION: security/controlled-document-procedure.md

Purpose

GitLab deploys control activities through policies and standards that establish what is expected and procedures that put policies and standards into action.

The purpose of this procedure is to ensure that there is consistency in developing and maintaining controlled documents at GitLab utilizing a hierarchal approach for managing legal and regulatory requirements.

There are two types of documentation at GitLab:

1. Controlled Documents: Formal policies, standards and procedures.
1. Uncontrolled Documents: Informal runbooks, certain handbook pages, guidelines, blog posts, templates, etc.

Everyone at GitLab is welcomed and encouraged to submit an MR to create or suggest changes to controlled documents at any time.

Scope

This procedure applies to all controlled documents developed in support of GitLab's statutory, regulatory and contractual requirements. Uncontrolled documents are dynamic in nature and not in scope of this procedure.

Roles & Responsibilities

Role	Responsibility
------	----------------

-----	-----
-------	-------

Security Compliance Team	Responsible for implementing and maintaining Security Policies and oversight of supporting standards and procedures as part of ongoing continuous control monitoring
--------------------------	--

Security Governance Team	Responsible for conducting annual controlled documents review
--------------------------	---

Security Assurance Management (Code Owners)	Responsible for approving changes to this procedure
---	---

Control Owners	Responsible for defining and implementing procedures to support Security
----------------	--

policies and standards |

Procedure

Definitions by Hierarchy

- Policy: A policy is a high-level statement of intent and defines GitLab's goals, objectives and culture. Statutory, regulatory, or contractual obligations are commonly the root cause for a policy's existence. Policies are designed to be centrally managed at the organizational level (e.g. Security Compliance Team or Legal & Ethics Compliance Team).
- Standard: Standards are mandatory actions or rules that give formal policies support and direction by providing specific details that enable policies to be implemented. Standards may take the form of technical diagrams.
- Procedure: Procedures are detailed instructions to achieve a given policy and, if applicable, supporting standard and provide step-by-step instructions to follow. Procedures are decentralized and managed by process/control owners where a security control is translated into a business process.

Creation

At minimum, controlled documents should cover the following key topic areas:

- Purpose: Overview of why the controlled document is being implemented.
- Scope: Who or what does the controlled document apply to.
- Roles & Responsibilities: Who is responsible for doing what. This should refer to departments or roles instead of specific individuals.
- Policy Statements, Standard or Procedure: The details.
- Exceptions: Define how exceptions to the controlled document will be tracked.
- References: Procedure documents should map back to a governing policy or standard, and may relate to one or more procedures or other uncontrolled documentation.

Publishing

Creation of, or changes to, controlled documents must be approved by management or a formally designated representative of the owning department as defined in the Code Owners file prior to publishing.

Most controlled documents will be published to our public facing handbook. However, if there is non public data included in the controlled document, it should be published via an internal facing only mechanism (e.g. an internal GitLab project or an internal only handbook page). Controlled documents should be accessible to all internal team members.

Handbook header

Controlled documents require a handbook frontmatter attribute for controlled documents to classify them. This attribute also renders a warning header.

Review

Controlled documents are required to be reviewed and approved on at least an annual basis. Controlled documents may be updated ad-hoc as required by business operations. Changes must be approved by a code owner of the controlled document prior to merge.

Reviewers of controlled documents are required to

1. Ensure that "say why not just what" transparency is easily understood in the description.

The title should be concise but clear on the what.

1. Ensure that announcements for team members are scheduled (Slack, company newsletter), and tick off the MR template task.

List of Controlled Documents

An accurate list of current controlled documents can be found here.

Exceptions

Exceptions to Controlled Documents must be tracked and approved by the Controlled Document approver(s) via an auditable format. An exception process should be defined in each document.

Information security considerations such as regulatory, compliance, confidentiality, integrity and availability requirements are most easily met when companies employ centrally supported or recommended industry standards. Whereas GitLab operates under the principle of least privilege, we understand that centrally supported or recommended industry technologies are not always feasible for a specific job function or company need. Exceptions from the aforementioned standard or recommended technologies is discouraged. However, it may be considered provided that there is a reasonable, justifiable business and/or research case for a Controlled Document exception; resources are sufficient to properly implement and maintain the alternative technology; the process outlined in this and other related documents is followed and other policies and standards are upheld.

In the event a team member requires an exception from the standard course of business or otherwise allowed by policy, the requester must submit an Exception Request to the Security Assurance Department, which contains, at a minimum, the elements outlined in the issue template.

Exception request approval requirements are documented within the issue template. The requester should tag the appropriate individuals who are required to provide an approval per the approval matrix.

If the business wants to appeal an approval decision, such appeal will be sent to Legal at legal@gitlab.com. Legal will draft an opinion as to the proposed risks to the company if the exception were to be granted. Legal's opinion will be forwarded to the CEO and CFO for final disposition.

Any exception approval must:

- Recommended compensating controls to reduce exposure and/or harm (i.e. admin rights to financially significant system may require audit logs and review of users activity within the system)

- Be captured in the associated exception request issue. All sections in the "Approvals" area of the issue must be filled in.

Once an exception request is submitted, the following general flow will commence:

- Requestor will receive approval from their Department head or the policy, standard, or procedure maintainers, depending on the type of exception request.
- Once approved by the requestor's Department, the team that owns the associated document will review the exception request.
 - Security Compliance will determine whether the exception request has any current or planned compliance implications.
 - The Controlled Document Owner will review the request to ensure appropriate compensating controls have been documented and will consider inputs from Security Compliance and other SMEs when determining the overall level of risk associated with the request.
- The Controlled Document Owner will document a final decision, and if appropriate, a recommended action plan to mitigate risk from the exception request.
- The exception will be logged in the central exception management space.
- The exception will be reviewed as its expiration date approaches, and any extension of the exception will require a new and approved extension request.

References

- GCF Compliance Controls
- Data Classification Standard
- Controlled Documents Work Instruction

SECTION: security/critical-projects.md

How are critical projects defined?

These projects aren't considered a function of OKRs (e.g., ambitious) but are considered critical because they

- must be accomplished fully within a specific time frame; and
- require cross-functional collaboration; and
- address immediate-term risks within security or GitLab

{{% alert title="Note" color="primary" %}}

Critical projects are DRI'd by a Senior Manager or higher, and approval must be given by functional leaders impacted by the critical project.

{{% /alert %}}

Critical Projects must use the label sec-okr::p0.

SMART and not an OKR

In essence, SMART goals are about setting clear, achievable objectives, while OKRs focus on setting ambitious goals to drive performance and align efforts across an organization.

A SMART goal is a framework for setting clear, attainable objectives, characterized by the acronym SMART, which stands for Specific, Measurable, Achievable, Relevant, and Time-bound:

Specific: The goal is well-defined and clear to anyone who has a basic knowledge of the project.

Measurable: The goal has criteria for measuring progress and completion, making it easy to track success.

Achievable: The goal is realistic and attainable, not something too far out of reach.

Relevant: The goal matters to the business or individual, aligning with broader objectives.

Time-bound: The goal has a deadline or defined timeframe, giving it a sense of urgency and a timeline for completion.

Relationship to Security OKRs

- Work toward these objectives takes a higher priority over OKRs
- The more critical projects we assume, the fewer OKRs we should have. It should be considered acceptable to have a quarter with one or zero OKRs due to critical project load.
- Expected critical projects are identified during the OKR planning process between functional leaders.
- It's possible for there to be zero critical projects in a given quarter.
- It's possible for an unexpected and critical project to appear mid-quarter outside of the OKR planning cycle. Functional leaders should recalibrate relevant KR expectations accordingly, and within the relevant OKR issues.

SECTION: security/customer-requests.md

We at GitLab are committed to helping our customers succeed. When a GitLab.com customer reaches out for assistance with a security incident that is the result of their implementation or use of GitLab, we will do our best to assist with the investigation by providing additional information surrounding the event, if available.

What type of assistance can GitLab provide to customers if they experience a security incident?

If a GitLab.com customer determines that the information available through the Audit Events feature doesn't provide sufficient information about activities taken on the customer's account, GitLab may provide reasonable assistance by performing additional log-dives under certain conditions and requirements.

To whom does this apply?

This applies to all paying GitLab.com SaaS customers on Premium or Ultimate plans. See conditions and requirements for more information.

Who is involved in this process?

Customer Success Managers (CSMs) and GitLab Support are our customers' main contacts at GitLab. These teams will ensure that the communication with the customer takes place as efficiently as

possible, will provide status updates regarding the log request, and ultimately hand out any artefacts that might result from the request to the customer.

GitLab Support handles non-complex application log requests that are within a 7-day time window from the time of the request, and which don't disclose Personal Data, such as user names and IP addresses.

GitLab's Security Incident Response Team handles complex, extensive requests according to an internal runbook for customer response operations. In order to maximize efficiency and provide results in a timely manner, customers will not be able to interface with the relevant security team directly during these requests. All communication with the customer is channelled through GitLab Support or the dedicated CSM.

Process outline

1. Customer contacts GitLab Support or CSM with a request that meets the required conditions and requirements.
1. GitLab Support or CSM triggers the workflow from Slack with the command /security to automatically create a confidential SIRT issue.
1. This newly created SIRT issue tags @gitlab-com/gl-security/sirt directly. The requesting customer must not be added to the issue.
1. Approval from GitLab Legal @lasayers should only be requested if an exception is required.
1. GitLab Support or CSM verifies that the requesting customer complies with our conditions and requirements for the request.
1. The relevant team (either GitLab Support or GitLab Security) starts working on the case, depending on the type of requests. The issue is updated with every new finding.
1. GitLab Support or CSM provides status updates to the customer based on the updates captured in the issue on a 12-hour/business day best-effort cadence.
1. Once the requested artefacts (i.e. log entries) are collected, GitLab Security and GitLab Legal review them.
1. Once reviewed and approved, GitLab Support or CSM share the artefacts with the requesting customer.
1. The team that performed the log-dive creates an issue with ~"Category:Audit Events" and ~"Enterprise Edition" labels to document the gap in Audit Events that forced the customer to request our assistance. This is to ensure that the missing functionality in the product is added in the future.

Conditions and requirements

Each request for assistance should comply with the conditions outlined in this section

Customer is on the GitLab Premium or Ultimate plan

The assistance outlined on this page only applies to paying GitLab.com customers on the Premium or Ultimate plans. It does not apply to free accounts, trial accounts, accounts on deprecated support plans, or accounts that are not subject to payment for other reasons.

Account scope and ownership

Requests for assistance can only apply to the GitLab.com account that the customer owns. GitLab

will not be providing assistance or information about GitLab.com accounts that the requesting customer does not own.

If a specific request applies to a GitLab.com account that is not owned by the requesting customer, a legal subpoena will be required. For such cases, the customer should reach out to GitLab Legal.

Time window and Scope of the log-dive request

At this time, GitLab is unable to perform log-dives exceeding a 1-month time window from the time of the request. The scope of the request should be specific and as narrow as possible. If it is determined by GitLab that the scope is too broad and excessive, GitLab may request for the scope to be narrowed, or inform the customer of the cost they will incur in performing the investigation.

Please note that because of the amount of logs that GitLab.com generates daily, log-dives exceeding a 7-day time window from the time of the request will take longer to complete.

List of relevant GitLab namespaces, projects and users

The request should contain a list of specific GitLab.com resources of interest - projects, users, groups. If the list is considered excessive, GitLab will ask for the request to be re-scoped. This is to ensure that the request is handled in a timely and resource-efficient manner.

Type of relevant events

The request should contain a list of specific GitLab.com events of interest. If the list is considered excessive, GitLab will ask for the request to be re-scoped. This is to ensure that the request is handled in a timely and resource-efficient manner.

Expectations

Receiving updates on the status of the request

Log-dives can be time consuming. The GitLab teams performing the log-dives will be providing updates through the designated issue. GitLab Support/Account Owner/CSM will relay the updates to the customer. Attempts to contact GitLab Security or its individuals on the status of the request will be unanswered. This applies to out-of-band communication as well (i.e. social media, personal e-mails, phone calls).

Access logs

Due to the volume of access logs generated daily on GitLab.com, we are currently not able to perform log-dives on access logs (i.e. HTTP requests) directly. Instead, we're relying on application logs and logs generated by other components of our production stack.

Information provided from logs should be redacted to ensure that we are not disclosing (i) GitLab Confidential or system information or (ii) any information pertain to accounts other than those subject to the request.

FAQ

What about GitLab's self-managed customers?

This process applies to GitLab.com only. GitLab is unable to produce logs for self-managed customers.

Why is this available only to GitLab.com customers that are on Premium or Ultimate plans?

GitLab Premium or Ultimate plans provide access to the Audit Events feature, which should provide the required information to help advanced customers to monitor the security and health of their GitLab.com accounts. This process is an extension of that feature that will:

- Help the customers get out of a tough situation.
- Ensure that the product provides functionalities that are important to customers.

SECTION: security/ecosystem.md

Overview

This page outlines the Security Division ecosystem, by describing the different processes of our departments.

These processes, represented with diagrams, highlight the data flows between our teams but also with external actors like the Product or the Engineering divisions.

Objectives

This page describe how to maintain the Security Division ecosystem.

Scope of the Security Ecosystem

Every process where Security is involved should be documented in this page. Each Security Department is represented and responsible for their own diagrams.

Process

Diagrams are maintained by the team or department responsible for the process described. The ecosystems should be available in each departments subfolders, so that the CODEOWNERS file is leveraged to require the right approvers. A dedicated page is recommended for each department, and even each sub-department.

Diagrams referring to non SAFE content can be kept in the internal handbook.

The Security Division Ecosystem should be maintained and updated as processes are updated to keep it as the single source of truth for these processes.

The Security Leadership will review the ecosystem diagrams prior to every offsite.

Tooling

In order to maintain consistency across all diagrams and this ecosystem in general, we use the Mermaid native integration in GitLab and in this Handbook.

Diagrams were created during the FY24-Q2 Security Leadership Offsite and are available in the Security Google Drive (only available for team members of the Security Division) until they are migrated in the handbook.

While sequence diagrams links are not yet supported, it can be useful to add links above or below ecosystem diagrams to runbooks or other documentation.

Guidelines

The Mermaid DSL for sequence diagrams already define how participants and interactions are declared. Use the following guidelines and recomm

Sequences numbers

Use sequence numbers (autonumber) to get a sequence number attached to each arrow. This helps the reading of the sequences and let us reference a part of the diagram if necessary.

Example:

```
mermaid
sequenceDiagram

    autonumber

    participant H1 as HackerOne
    participant SE as SecEng

    H1->>SE: Hackerone report
    SE-->>H1: Validation
```

Alternative paths

You can use alternative paths (alt) to express "or" conditions.

Example:

```
mermaid
sequenceDiagram

    autonumber

    participant H1 as HackerOne
    participant TM as Threat Management
    participant SE as SecEng
```



```
alt Report/Research/Threat
  TM->>SE: Threat
else
  H1->>SE: Hackerone report
end
```

Resources

- Mermaid sequence diagrams documentation
- FY24-Q2 Offsite: Ecosystem diagrams issue related to the creation of this page.

SECTION: security/engaging-with-security.md

Vulnerability Reports and HackerOne

GitLab receives vulnerability reports by various pathways, including:

- HackerOne bug bounty program
- Reports or questions that come in from customers through Zendesk.
- Issues opened on the public issue trackers. The security team can not review all new issues and relies on everyone in the company to identify and label issues as ~bug::vulnerability and @-mention @gitlab-com/gl-security/product-security/appsec on issues.
- Issues reported by automated security scanning tools

For any reported vulnerability:

- Open a confidential issue in the appropriate issue tracker as soon as a report is verified. If the vulnerability was reported via a public issue, make the issue confidential. If triage is delayed due to team availability, the delay should be communicated.
- Add ~security and ~bug::vulnerability labels to the issue. Add the appropriate group label if known.
- Add the ~Weakness::CWE-XXX label, where the XXX is the weakness related CWE ID.
- An initial determination should be made as to severity and impact. Never dismiss a security report outright. Instead, follow up with the reporter, asking clarifying questions.
- For next steps, see the process as it is detailed below for HackerOne reports, and adhere to the guidelines there for vulnerabilities reported in other ways as well in terms of frequency of communication and so forth.
- Remember to prepare patches, blog posts, email templates, etc. following the security engineer process or in other non-public ways even if there is a reason to believe that the vulnerability is already out in the public domain (e.g. the original report was made in a public issue that was later made confidential).

Triage Rotation

See the dedicated page to read about our Triage Rotation process.

HackerOne Process

See the dedicated page to read about our HackerOne process.

Security Dashboard Review

See the dedicated page to read about our dashboard review process.

CVE IDs

We use CVE IDs to uniquely identify and publicly define vulnerabilities in our products. Since we publicly disclose all security vulnerabilities 30 days after a patch is released, CVE IDs must be obtained for each vulnerability impacting self-managed to be fixed. The earlier obtained the better, and it should be requested either during or immediately after a fix is prepared.

We currently request CVEs through our CVE project. Keep in mind that some of our security releases contain security related enhancements which may not have an associated CVE or vulnerability. These particular issues are not required to obtain a CVE since there's no associated vulnerability.

On Release Day

On the day of the security release several things happen in order:

- The new GitLab packages are published.
- All security patches are pushed to the public repository.
- The public is notified via the GitLab blog release post, security alerts email, and Twitter.

The GitLab issue should then be closed and - after 30 days - sanitized and made public. If the report was received via HackerOne, follow the HackerOne process.

Process for disclosing security issues

At GitLab we value being as transparent as possible, even when it costs. Part of this is making confidential GitLab issues about security vulnerabilities public 30 days after a patch. The process is as follows:

1. Check for a ~keep confidential tag. If one exists
 1. Decide whether this tag is still appropriate and in line with our Transparency value
 1. Start a discussion with issue participants, if needed
1. If an issue does not have ~keep confidential, remove sensitive information from the description and comments, e.g.
 1. Proof-of-concept videos & screenshots showing researcher account information
 1. Tokens, Access Keys, and other secrets
 1. Information which our Data Classification Standard and SAFE framework say to not disclose
1. Issues related to personal data leaks are not disclosed since they are not security issues related to the product. If for some reason it needs to be disclosed then consult with Legal and the Corporate Comms team before disclosing.
1. Identify all issue description changes, click to expand "Compare with previous version" and

click the trash icon to "Remove description history"

1. Optionally mention issue participants to notify them you intend to make the issue public
1. Optionally add a comment that explains the CVSS.
1. Edit the Confidentiality of the issue and set it to Public

To facilitate this process the GitLab Security Bot comments on confidential issues 30 days after issue closure when they are not labeled ~keep confidential.

Handling Disruptive Researcher Activity

Even though many of our 3rd-party dependencies, hosted services, and the static about.gitlab.com site are listed explicitly as out of scope, they are sometimes targeted by researchers. This results in disruption to normal GitLab operations. In these cases, if a valid email can be associated with the activity, a warning such as the following should be sent to the researcher using an official channel of communication such as ZenDesk.

text

Dear Security Researcher,

The system that you are accessing is currently out-of-scope for our bounty program or has resulted in activity that is disruptive to normal GitLab operations. Reports resulting from this activity may be disqualified from receiving a paid bounty. Continued access to this system causing disruption to GitLab operations, as described in policy under "Rules of Engagement, Testing, and Proof-of-concepts", may result in additional restrictions on participation in our program:

Activity that is disruptive to GitLab operations will result in account bans and disqualification of the report.

Further details and some examples are available in the full policy available at:

<https://hackerone.com/gitlab>

Best Regards,

Security Department | GitLab

<https://handbook.gitlab.com/handbook/security/>

Creating New Security Issues

Use the Vulnerability Disclosure issue template to report a new security vulnerability, or use our HackerOne bug bounty program. Please note that we are only able to pay out bounties for eligible vulnerabilities through the HackerOne platform.

New security issue should follow these guidelines when being created on GitLab.com:

- Create new issues as confidential if unsure whether issue a potential vulnerability or not. It is easier to make an issue that should have been public open than to remediate an issue that should have been confidential. Consider adding the /confidential quick action to a project issue template.
- Always label as ~security at a minimum. If you're reporting a vulnerability (or something you suspect may possibly be one) please use the Vulnerability Disclosure template while creating the issue. Otherwise, follow the steps here (with a security label).
- Add any additional labels you know apply. Additional labels will be applied by the security team and other engineering personnel, but it will help with the triage process:
 - ~"type::bug", ~"type::maintenance", or ~"type::feature" if appropriate
 - Team or DevOps lifecycle labels
 - ~customer if issue is a result of a customer report
 - ~internal customer should be added by team members when the issue impacts GitLab operations.
 - ~dependency update if issue is related to updating to newer versions of the dependencies GitLab requires.
 - ~featureflag:: scoped labels if issue is for a functionality behind a feature flag
- Issues that contain customer specific data, such as private repository contents, should be assigned ~keep confidential. If possible avoid this by linking resources only available to GitLab team member, for example, the originating ZenDesk ticket. Label the link with (GitLab internal) for clarity.

Occasionally, data that should remain confidential, such as the private project contents of a user that reported an issue, may get included in an issue. If necessary, a sanitized issue may need to be created with more general discussion and examples appropriate for public disclosure prior to release.

For review by the Application Security team, @ mention @gitlab-com/gl-security/product-security/appsec.

For more immediate attention, refer to Engaging security on-call.

Severity and Priority Labels on ~security Issues

Severity and priority labels are set by an application security engineer at the time of triage if and only if the issue is determined to be a vulnerability.

To identify such issues, the engineer will add the ~bug::vulnerability label.

Severity label is determined by CVSS score, using the GitLab CVSS calculator.

If another team member feels that the chosen ~severity / ~priority labels need to be reconsidered, they are encouraged to begin a discussion on the relevant issue.

The presence of the ~bug::vulnerability label modifies the standard severity labels(~severity::1, ~severity::2, ~severity::3, ~severity::4)

by additionally taking into account

likelihood as described below, as well as any

other mitigating or exacerbating factors. The priority of addressing

~security issues is also driven by impact, so in most cases, the priority label assigned by the security team will match the severity label.

Exceptions must be noted in issue description or comments.

The intent of tying ~severity/~priority labels to remediation times is to measure and improve GitLab's response time to security issues to consistently meet or exceed industry standard timelines for responsible disclosure. Mean time to remediation (MTTR) is a external metric that may be evaluated by users as an indication of GitLab's commitment to protecting our users and customers. It is also an important measurement that security researchers use when choosing to engage with the security team, either directly or through our HackerOne Bug Bounty Program.

Vulnerabilities must be mitigated and remediated according to specific timelines. The timelines are specified in the Vulnerability Management handbook (a controlled document).

If a better understanding of an issue leads us to discover the severity has changed, recalculate the time to remediate from the date the issue was opened. If that date is in the past, the issue must be remediated on or before the next security release.

Due date on ~security Issues

For ~security issues with the ~bug::vulnerability label and a severity of ~severity::3 or higher, the security engineer assigns the Due date, which is the target date of when fixes should be ready for release. This due date should account for the Time to remediate times above, as well as monthly security releases on the 28th of each month. For example, suppose today is October 1st, and a new severity::2 ~security issue is opened. It must be addressed in a security release within 30 days, which is October 31st. So therefore, it must catch the October 28th security release. Furthermore, the Security Release Process deadlines say that all merge requests associated with the fix must be ready 48 hours before the due date of the security release, which would be October 26th. So the due date in this example must be October 26th.

Note that some ~security issues may not need to be part of a product release, such as an infrastructure change. In that case, the due date will not need to account for monthly security release dates.

On occasion, the due date of such an issue may need to be changed if the security team needs to move up or delay a monthly security release date to accommodate for urgent problems that arise.

Scheduling ~security Issues

Product Managers and Engineering Managers should follow the recommended guidance when scheduling ~security Issues :

When a team is assigned an	This is the expected response

| S1 | Disrupt your milestone and work on the ~"bug::vulnerability" and ~"FedRAMP::Vulnerability" security issue right away |

| S2 | Disrupt your milestone and work on the ~"bug::vulnerability" and ~"FedRAMP::Vulnerability" security issue right away |

| S3 | Begin working on the ~"bug::vulnerability" and ~"FedRAMP::Vulnerability" security issue at the beginning of the next Milestone